Multi-party Computation based Location Services

Kanav Gupta
University of Maryland

Amadeo De La Vega
University of Maryland

Harikesh Kailad
University of Maryland

***Abstract*—Nearest neighbor search is an important building block in many modern tools and applications. However, modern day applications also often require more security and privacy guarantees. With the development of secure multiparty computation, we now have protocols which allow users to do computation without revealing data. We investigate the problem of private nearest neighbor search; where clients can make nearest neighbor queries to a remote database, but neither the client learns anything about the database nor the server learns anything about the query.**

While private nearest neighbor protocols exist, existing protocols are inefficient and can only respond with *approximate* results. In this work, we utilize recent developments in MPC, specifically *function secret sharing*, to implement a concretely efficient nearest neighbor search. Our protocol runs in the three-server model to ensure database privacy. We implement and evaluate this protocol, and report its performance.

1. Introduction

Nearest Neighbor Search is an important algorithmic problem with many applications. The problem asks to find the point $y \in \mathbb{R}^d$ from a set of points $\mathcal{D}$ closest to a target point $x \in \mathbb{R}^d$. Although the problem itself does not require any privacy or security guarantees, in almost all applications of nearest-neighbor search, there are privacy requirements and risks associated with the database or the vector.

In many *natural language processing* (NLP) algorithms, one must find a word from a set of words such that it's embedding is close to a specifically generated target embedding. To do this, nearest neighbor search is used. However, for NLP models deployed in practice, the dataset and the target embedding come from two different parties, and both data objects contain highly sensitive inputs from the parties. Hence, it would be ideal to perform this operation without sharing private data with each other. Additionally, in smart *information retrieval* systems, nearest neighbor search is used to find a document from a set, owned by a service provider, which is closest to a set of keywords provided by the client. In this application, the keywords sent by the user might contain private information. In machine learning and information retrieval, privacy-preserving techniques have already garnered interest, with implementations of private LLM search [31].

In a more general version of private nearest neighbor search, we are tasked to find *all* points $y \in \mathbb{R}^d$ in the database $\mathcal{D}$ such that, for some distance metric dist and a threshold distance t, it holds that $\mathsf{dist}(x, y) \leq t$. This variant is also useful in many scenarios. In many *battle-royale games* (e.g., Fortnite, CSGO, etc.), each player constantly needs to learn the position of all players that are nearby. In the case where one does not have a trusted server to host the positions of all players, it is non-trivial to realize the system without a cheating user. Similarly, in *ride-sharing apps* like Uber, nearest-neighbor search (on graphs) is used to find the nearest driver. However, users may not want to reveal their geographical location to a single company or server. In this setting, users may only want to share their geographical location with the drivers who are nearby.

This problem of privacy can be solved using a specialized *secure multi-party computation* (MPC) protocol [12], [30] that performs *private nearest neighbor search*. MPC allows a set of mutually distrusting parties to perform a computation over their private inputs. Security of MPC guarantees that the parties don't learn anything other than the computation output. Many previous works [19], [24], [25], [27] addressed the problem of private nearest neighbor search, but all of them only provide approximate correctness. All of them claim that computing exact nearest neighbor search using MPC would be infeasible and would incur significant costs.

Recent advancements in MPC shows great promise. Specifically, a recent branch of protocols, called *Function Secret Sharing (FSS) based Protocols* [3], [15], [16], [17], [23], [26], [28], have shown potential to result in protocols with low latency and communication requirements. With this in mind, we try to verify the claim made by prior works on approximate nearest neighbor search and seek the answer to following question:

Can private exact nearest neighbor search, implemented using recent advancements in FSS-based protocols, outperform prior work on private approximate nearest neighbor search?

Contributions. We construct protocols for private nearest neighbor search using function secret sharing based protocols as a black box. In addition, we implement and evaluate our protocol. We test the performance of this protocol with both private and public databases, and compare this performance with state-of-the-art prior work. We find that our protocol outperforms the prior state-of-the-art Preco [24], even when their system work on a smaller database and

more powerful servers.

Organization. We first discuss related work in this field of multiparty computation. We then describe our threat model and the primitives used: secret sharing and function secret sharing. We describe our protocol for performing generalized nearest neighbors with public and private databases. We then describe how we can perform this protocol privately in MPC using FSS-based protocols. Finally, we implement and evaluate the end-to-end protocol for generalized nearest neighbor search.

2. Related Works

Homomorphic Encryption based k-NNs. Homomorphic encryption offers one solution to the private nearest neighbors problem, where a single server holds encrypted data, and clients can query and compute the distances in private. While this line of work has led to constructions of multi-key FHE and multiple implementations of private nearest neighbors [6], [7], [18], homomorphic encryption is not practical enough for lighweight servers to answer queries for hundreds of clients with a large number of entries.

In **Preco** [24], Servan-Schreiber et al. construct a system for private nearest neighbor search and measurement through private information retrieval techniques. The authors utilize a LSH based data structure to encode data, and search for nearest neighbors with a privacy preserving protocol. This protocol employs *distributed point functions* and *oblivious masking* rather than generic powerful cryptographic techniques, which allows for a practical implementation. The Preco protocol runs on the 2-server model, with secret shares of data held between the servers. While elegant and more efficient than FHE or generic MPC approaches, Preco does not achieve the level of practical efficiency necessary for moderately sized parties to host private databases. When servers are hosted by large corporations such as Google or Amazon, Preco can run private nearest neighbors extremely quickly even for lightweight clients. However, for lightweight servers (such as when running a game server), this is not the case. Furthermore, Preco does not utilize more modern function secret sharing protocols, which can improve performance. Preco additionally only provides approximate answers (and so solves the approximate nearest neighbors problem).

FSS based MPC. Many modern private computation protocols [15], [16] now rely on *function secret sharing* [4], a new primitive generalizing DPFs introduced by Gilboa et al. FSS-based protocols allow clients to generate preprocessing to perform specific computations. Specifically, in our setting, we are interested in computing private comparisons efficiently. Function secret sharing allows clients to perform comparisons extremely efficiently, which is extremely useful for testing vectors against a threshold and finding close vectors. Function secret sharing has been used to improve the performance of multiple building blocks [5] in multiparty computation, which can help with image and regex matching, nearest neighbors, and more.

Private Single Server ANNs. Additionally, a line of prior work focus on the approximate nearest neighbor problem in the single server setting [19], [25], [27], where either FHE is required and the protocol is not practically efficient or some privacy is leaked. These protocols also do not answer with an exact match, but only an approximate match. While ANN is also useful in many applications, exact matches are typically necessary for information retrieval.

3. Preliminaries

3.1. Notation

We use λ to denote the computational security parameter. Whenever n is clear from the context, we let $N = 2^n$. We use $\mathbb{Z}_N$ to denote the ordered set of n-bit unsigned integer, i.e. $\{0, 1, \ldots N-1\}$. We let $\mathbb{G}$ denote a finite group (written additively). In context of nearest neighbor search, we use d to represent the dimensionality of the vectors and m to represent the number of vectors in the database. We represent each vector element in $\mathbb{Z}_N$ where $n = 64$. In case we need real-valued elements, we use fixed-point representation to represent them in $\mathbb{Z}_N$. For a vector $\vec{x} \in \mathbb{Z}_N^d$, we use $\vec{x}[i]$ to represent the i-th element, in 0-indexing. We use $\langle \vec{x}, \vec{y} \rangle \in \mathbb{Z}_N$ to denote the inner product of $\vec{x}$ and $\vec{y}$. For a predicate p, we use $1\{p\}$ to denote the boolean value of 1 if p is true, and 0 otherwise.

3.2. Threat Model

In this work, we consider a threat model containing multiple clients and three non-colluding servers. We assume the servers to be honest-but-curious. That is, they follow the protocol instructions as per the transcript, but may attempt to extract as much information as possible. On the other hand, we place no restrictions on the clients, i.e. they can behave arbitrarily.

3.3. Secret Sharing

Two-party secret sharing allows a dealer to split a secret value $x \in \mathbb{G}$ into two shares x_0, x_1, such that neither share reveals anything about x, yet the shares jointly allow for reconstruction of x; each share can then be given to a different party. In our work, we use additive secret sharing scheme. In this scheme, we pick x_0 uniformly from $\mathbb{G}$ and set $x_1 = x - x_0$. In this work, the values being shared would be either be integer ($\mathbb{G} = \mathbb{Z}_N$) or boolean ($\mathbb{G} = \{0, 1\}$). To secret share a vector, we secret share the individual elements of the vector.

Additive secret shares also support linear homomorphism. That is given public values $a, b \in \mathbb{G}$ and secret shares (x_0, x_1) of x held by P_0 and P_1, the two parties can compute shares of $y = ax + b$ non-interactively. This can be achieved by P_0 computing $y_0 = ax_0$ and P_1 computing $y_1 = ax_1 + b$.

3.4. Secret Sharing based MPC

Starting from the seminal work of Goldreich, Micali, and Wigderson [12], a large line of multi-party computation protocols [2], [8], [9], [10], [11], [20], [21] operate over secret shares. In these protocols, the circuit is first split into several building blocks or *gates*. Each gate can have arbitrary number of fan-ins or fan-outs, and performs some operation, e.g. addition, multiplication, comparison etc. For each of these operations, these protocols provide a secure MPC sub-protocol that takes as input secret shares of the inputs and produces the secret shares of the correct operation output. For example, in the sub-protocol for integer multiplication, provided by a two-party computation protocol, the two parties would input x_0, y_0 and x_1, y_1 respectively, where (x_0, x_1) are secret shares of x and (y_0, y_1) are secret shares of y. The sub-protocol would output z_0 to P_0 and z_1 to P_1, such that $z_0 + z_1 = x \cdot y = (x_0 + x_1) \cdot (y_0 + y_1)$. Such protocol design allow for arbitrary composition: given a secret-sharing based protocol for a set of operations, any circuit can be computed securely, as long as the circuit can be decomposed into operations from the set.

3.5. Function Secret Sharing

A two-party Function Secret Sharing (FSS) scheme for a class of functions $\mathcal{F}$ allows a dealer to split a function $f \in \mathcal{F}$ into two functions f_0, f_1, such that for all inputs x, $f_0(x) + f_1(x) = f(x)$. Additionally, each of f_0, f_1 do not reveal any information about f. We now formally define FSS based on definition from [4]:

Definition 1 (Function Secret Sharing)**.** *A (2-party) FSS scheme for function family $\mathcal{F} = \{f : \{0,1\}^* \to \mathbb{G}\}$ is a pair of efficient algorithms* $(\mathsf{Gen}, \mathsf{Eval})$ *such that*

- $\mathsf{Gen}(f)$ *is a randomized algorithm that outputs function keys* (k_0, k_1).
- $\mathsf{Eval}(k_b, x)$ *is a PPT algorithm that takes as input a key k_b and an input $x \in \{0,1\}^*$, and outputs $y_b \in \mathbb{G}$.*

Correctness requires that for all $f \in \mathcal{F}$ and $x \in \{0,1\}^*$, if $(k_0, k_1) \leftarrow \mathsf{Gen}(f)$, then $\mathsf{Eval}(k_0, x) + \mathsf{Eval}(k_1, x) = f(x)$. Security requires existance of an efficient simulator $\mathcal{S}$ such that for every function f and $b \in \{0,1\}$, the output of the following two experiments are computationally indistinguishable:

- *Run $(k_0, k_1) \leftarrow \mathsf{Gen}(f)$ and output k_b.*
- *Output $\mathcal{S}(1^\lambda, b)$.*

3.6. FSS-based 2PC in Preprocessing Model

Multiple prior works [3], [15], [16], [17] have used FSS to devise 2PC protocols with extremely fast online phase in the preprocessing model [9]. We now unpack the above statement to understand the impact of FSS.

Many works in MPC work in preprocessing model. In this model, prior to recieving the actual computation inputs, the parties engage in a input-independent preprocessing phase to learn *correlated randomness*. In the online phase, the parties recieve the input shares, and use the correlated randomness generated in the preprocessing phase to realize a fast MPC. FSS enables design a design of protocols for which the cost of performing the online phase is extremely low. We call such protocols *FSS-based Protocols*.

In the literature, to implement the preprocessing phase, two methods have been considered. One option is to use a standard MPC protocol (i.e. that works without preprocessing phase) over the function that generates preprocessing material to get the required correlated randomness. Using this approach, the preprocessing phase usually costs much more than the online phase. In fact, to generate correlated randomness for FSS-based protocols, the best known protocols are exponentially more expensive the online phase. An alternative approach would be to introduce a new non-colluding semi-honest party that generates the correlated randomness honestly and distributes among the original parties. In FSS-based protocols, prior works use this approach to perform the preprocessing phase due to large costs of the first approach, at the cost of introducing an additional party.

4. Our End-To-End Protocol Design

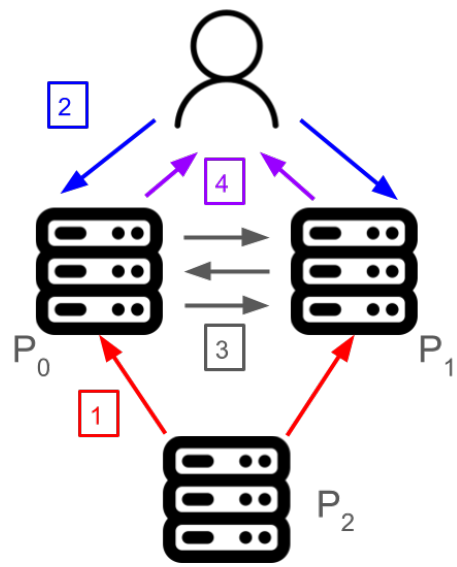

Figure 1. Summary of our overall protocol. Our protocol proceeds in four steps. (1) P_2 provides preprocessing material to P_0 and P_1 (2) Client provides shares of input vector to P_0 and P_1 (3) P_0 and P_1 perform fast online phase of the protocol (4) P_0 and P_1 send the output shares to client.

We implement our protocol for secure nearest neighbor search using three non-colluding servers. Prior to recieving input from the client, one of the three parties, P_2, generates the required FSS-correlations and distributes it among the other two parties, P_0 and P_1. When client needs to perform the computation on its input, it secret shares the input among P_0 and P_1. The two servers use the shares and the correlated randomness, provided by P_2, to generate the shares of the required output value. The parties send these shares to the client, who adds up the shares to get the required output. The flow is summarized in Figure 1.

We consider two variants of the nearest neighbor search problem, (1) server-public database with single nearest vector output, and (2) server-private database with multiple outputs. In the coming subsections, we describe the underlying computation in detail. Although we only consider these two variants, other variants can also be implemented using slight modifications. To realize a secure protocol to carry out these computations, the servers can simply replace calls to a building block with its corresponding protocols (described in Section 5).

4.1. Server-Public Database with Single Nearest Vector Output

In this setting, we assume that the three servers hold a copy of the database of vectors. Starting with the (secret shares of) target vector, the servers need to compute (secret shares of) the closest vector in the database.

To do this, the servers first calculate the distance of the target vector with all vectors in the database. That is, for target vector $\vec{x} \in \mathbb{Z}_N$ and each database vector $\vec{y}_i \in \mathcal{D}$:

$$\begin{aligned} a_i &= \sum_{j=0}^{d-1} (\vec{x}[j] - \vec{y}_i[j])^2 \\ &= \Big(\sum_{j=0}^{d-1} \vec{x}[j]^2\Big) - 2\langle \vec{x}, \vec{y}\rangle + \Big(\sum_{j=0}^{d-1} \vec{y}_i[j]^2\Big) \end{aligned}$$

Now, we need to find $\vec{y}_i$ with the smallest a_i. As all $\vec{y}_i$ are known to ther servers, they can precompute the third term (i.e. $\sum_{j=0}^{d-1} \vec{y}_i[j]^2$) for each $\vec{y}_i$. Also, we can skip the computation of the first term (i.e. $\sum_{j=0}^{d-1} \vec{x}[j]^2$) as it is a common additive term in all a_is. Hence the problem reduces to computation of $\langle \vec{x}, \vec{y}_i\rangle$. Again, as $\vec{y}_i$ is public, this is an inner product between a secret shared vector and public vector, hence is free of cost. Hence overall, computing secret shares of distances is *free of communication* in this setting.

Now given set of distances $\{a_0, a_1 \ldots a_{m-1}\}$, we need to compute the index k of the smallest distance. That is:

$$k = \mathsf{argmin}_i a_i$$

This operation can be done in a binary tree fashion. In the m leaves of the tree, we put the tuple (i, a_i) at the i-th leaf. Then, for each pair of adjacent tuples (i, a_i) and $i+i, a_{i+1}$, we compute the comparison bit $c = 1\{a_i < a_{i+1}\}$. We compute the parent of this node as a selection betweem (i, a_i) (if $c = 1$) and $(i+1, a_{i+1})$ (if $c = 0$). We repeat this step recursively until one node (k, a_k) remains. a_k part of the tuple can be dropped. In summary, this step requires protocols for comparisons and selections to implement.

Now given the index k, we need to fetch $\vec{y}_k$ from the database to compute the final required output. This operation is known as a table lookup. Hence, it can be implemented using a protocol for lookup table.

4.2. Server-Private Database with Multiple Vector Outputs

In this setting, we assume that the two out of the three servers (P_0 and P_1) hold secret shares of the database of vectors. Starting with the (secret shares of a) target vector $\vec{x}$, the servers need to compute (secret shares of) all vectors $\vec{y}_i$ in the database for which the distance between the $\vec{x}$ and $\vec{y}_i$ is less than a threshold t.

To do this, the servers, like previous setting, compute distance between the target vector and all vectors in database. In this setting however, the database vectors are also secret shared. So, the multiplications are not free, unlike the previous model. Restating the distance equation:

$$a_i = \Big(\sum_{j=0}^{d-1} \vec{x}[j]^2\Big) - 2\langle \vec{x}, \vec{y}\rangle + \Big(\sum_{j=0}^{d-1} \vec{y}_i[j]^2\Big)$$

The first term is known to the client, and can provide it's secret shares along with the shares of $\vec{x}$. In case the database vectors also are sourced from other clients (as in the battle royale application), the servers can also retrieve the secret shares of the third term from other clients. To calculate the second term, we require inner product of $\vec{x}$ and $\vec{y}$. To do this, we use the protocol for inner product (Section 5.1).

Once all m (squared) distances $\{a_0, a_1, \ldots a_{m-1}\}$ are calculated, for each a_i we compute the $s_i = 1\{a_i < t^2\}$. s_i is a bit which denotes whether $\vec{x}$ and $\vec{y}_i$ are close enough. This can be implemented using protocol for comparison (Section 5.2). Finally, using the bits s_i, we select between a special character $\perp$ and $\vec{y}_i$ to learn $\vec{z}_i$ using the protocol for selection. We send the shares of all $\vec{z}_i$ to the client.

5. Building Blocks

Due to Section 4, we now know that we only require 4 protocols to implement both variants of nearest neighbor search: multiplication, comparison, selection and lookup table. In the coming sections, we provide the relevant references from where we get our protocol for all of these building blocks.

5.1. Multiplication

We implement protocol for multiplication of two secret shared values using Beaver triples [1]. Consider that the two parties P_0 and P_1 want to multiply shares of $x\mathbb{Z}_N$ and $y \in \mathbb{Z}_N$. That is, P_0 holds x_0 and y_0, while P_1 holds x_1 and y_1, and they want to get shares of $z = x \cdot y$. To achieve this, P_2 generates random values $a, b \in \mathbb{Z}_N$ and calculates $c = a \cdot b$. It secret shares a, b, c among P_0 and P_1 in the preprocessing phase. In the online phase, each party P_σ ($\sigma \in \{0, 1\}$) computes $e_\sigma = x_\sigma - a_\sigma$ and $f_\sigma = y_\sigma - b_\sigma$ and opens these shares to learn $e = x - a$ and $f = y - b$. Now P_0 computes $z_0 = e \cdot f + a_0 \cdot f + e \cdot b_0 + c_0$ and P_1 computes $z_0 = a_1 \cdot f + e \cdot b_1 + c_1$. As $z_0 + z_1 = e \cdot f + a \cdot f + e \cdot b + c = (e + a)(f + b) = x \cdot y$, (z_0, z_1) form the secret shares they

need. This protocol can be extended to inner products by sampling vectors $\vec{a}$ and $\vec{b}$, instead of integers and replacing all multiplication operations above with inner products. We summarize the communication costs of the above protocol in the following theorem:

Theorem 1. *Consider secure inner product of two secret shared vectors $\vec{x}, \vec{y} \in \mathbb{Z}_N^d$. There exists a protocol that requires P_2 to communicate $n \cdot (2d+1)$ bits of correlated randomness to each party P_0 and P_1 in the preprocessing phase. In the online phase, it requires communication of $2nd$ bits from P_0 and P_1 each.*

5.2. Comparison

Given secret shares of two integers $x, y \in \mathbb{Z}_N$, our computation requires computing secret shares of a bit $b \in \{0, 1\}$ such that $b = 1\{x < y\}$. We use an FSS-based protocol for comparison from Sigma [15] to perform comparison of two secret shared values. We skip the complete description and summarize the costs of the protocol:

Theorem 2. *Consider secure comparison of two secret shared integers $x, y \in \mathbb{Z}_N$. There exists a protocol that requires P_2 to communicate $\approx (n-7)\lambda$ bits of correlated randomness to each party P_0 and P_1 in the preprocessing phase. In the online phase, it requires communication of n bits from P_0 and P_1 each.*

5.3. Selection

Given secret shares of a group element $x \in \mathbb{G}$ and a bit $s \in \{0, 1\}$, our computation requires computing secret shares of a group element $y \in \mathbb{G}$ such that $y = x$ if $b = 1$ and $y = 0$ otherwise. We use an FSS-based protocol for selection from Orca [17] to perform this operation. We skip the description and summarize the costs of the protocol:

Theorem 3. *Consider secure selection of a secret shared group element in $\mathbb{G}$. There exists a protocol that requires P_2 to communicate $4|\mathbb{G}|$ bits of correlated randomness to each party P_0 and P_1 in the preprocessing phase. In the online phase, it requires communication of $|\mathbb{G}|+1$ bits from P_0 and P_1 each.*

5.4. Lookup Table

Given secret shares of an n-bit integer i and a public table T containing 2^n elements of $\mathbb{Z}_N$, a protocol for lookup table computes the shares of the element $T[i]$. We use the protocol for lookup table from Pika [28]. We skip the description and summarize the costs of the protocol:

Theorem 4. *Consider secure lookup table of a secret shared integers $i \in \mathbb{Z}_N$ over a public table. There exists a protocol that requires P_2 to communicate $\approx (n-7)\lambda$ bits of correlated randomness to each party P_0 and P_1 in the preprocessing phase. In the online phase, it requires communication of n bits from P_0 and P_1 each.*

6. Implementation

We implement our protocol in C++ in 4000 lines of code. We leverage Eigen [13] to perform fast linear operations and use pseudorandom generator implementation from cryptoTools [22], required in FSS implementation. We implement FSS constructs of Distributed Point Function using native AES-NI instructions and use optimizations from Splinter [29] to reduce AES costs. We further optimize computation costs of lookup tables using early-termination optimization [4] and half-tree optimization [14]. The code for the implementation will be made public.

7. Evaluation

We evaluate our protocols to implement two variants of nearest-neighbor search: (1) Server-public database with single nearest vector output (Section 4.1) (2) Server-private database with multiple vector outputs (Section 4.2).

System Setup. We deploy the three servers on Azure East US, each using a D4s v5 instance, equipped with 16 GB of RAM. Our experiments run each server on single threads. All three servers are connected on LAN with a bandwidth of 10 Gbps and a round trip time of 0.1 ms. These servers cost ≈ 130 USD per month per server. We use a MacBook Pro 2019 model with an 4 core Intel Chip as the client located in College Park, MD. The connections between the client and the servers have a bandwidth of 20 Mbps and a round trip time of 40 ms.

7.1. Server-Public Database with Single Nearest Vector Output

In this application, we consider a database containing $m = 2^{18}$ vectors of $d = 5$ dimensions. Each element of a vector is a 64-bit integer. Our implementation in this setup requires a latency of 1.5 seconds per query.

Note that the previous work of Preco [24] also works in the same model, but there are a few fundamental differences. First, they consider *approximate* nearest neighbor search and are only correct *most* of the time, not all. Moreover, unlike us, their protocols cannot be extended to the setting where the database is private. Finally, they require only 2 servers, while we require a third server to take the role of dealer. While it would have been ideal to run the exact same setup on our machines, we were unable to do so due to time constraints and errors in the compilation of their implementation. Nonetheless, we refer to the evaluation section of their paper to compare with us. Based on Figure 9, fourth graph in [24], the setting with $m = 60,000$, single threaded, 100 probes and 20 hash tables, they require 1.6 seconds per query. This achieves an accuracy of 95%. Note that Preco requires a larger amount of time for lesser number of vectors. Moreover, they also make use of expensive c5.9xlarge AWS servers with 8000 CPUs and 72 GB of RAM, while we perform our evaluation on moderately sized servers.

7.2. Server-Private Database with Multiple Vector Outputs

In this application, we consider a database containing $m = 128$ vectors of $d = 3$ dimensions. Each element of a vector is a 64-bit integer. These parameters have been picked keeping battle-royale games (like Fortnite) in mind, where 100 players are located on a map and their position can be three dimensional (yes they can fly!). In this setting, our implementation requires 0.02 seconds per query. Assuming all players contact server continuosly, this implementation allows a player to make 1 query per 2 seconds, which we feel is acceptable. Note that this latency is entirely due to the communication between client and the servers. We believe this can be optimized in the future using a production grade communicators, which are used in modern games to achieve a much faster latency.

8. Conclusion

In this project, we considered the problem of exact private nearest neighbor search. We were able to demonstrate that our implementation, equipped with the recent advancements in FSS-based protocols, is able to beat the prior state of the art Preco [24]. Our implementation is also general enough to extend to other applications like battle-royale games and we demonstrate that our implementation provides acceptable latency to be used in games such as CSGO. In the future, we would like to explore the problem of 3 server *approximate*, nearest neighbor search (like Preco) using FSS. We would also like to improve our implementation to further reduce the latency. Additionally, we would also like to consider other models of security; while currently we need three servers with one acting as a dealer, it would be interesting to consider the setting with two servers, or where the servers may not be semi-honest but can be malicious.

References

[1] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, '91.

[2] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In *EUROCRYPT*, 2011.

[3] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *EUROCRYPT*, 2020.

[4] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In *CCS*, 2016.

[5] Elette Boyle, Lisa Kohl, Zhe Li, and Peter Scholl. Direct FSS constructions for branching programs and more from PRGs with encoded-output homomorphism. Cryptology ePrint Archive, Paper 2024/192, 2024.

[6] Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder V. L. Pereira. SortingHat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. Cryptology ePrint Archive, Paper 2022/757, 2022.

[7] Kelong Cong, Robin Geelen, Jiayi Kang, and Jeongeun Park. Revisiting oblivious top-k selection with applications to secure k-NN classification. Cryptology ePrint Archive, Paper 2023/852, 2023.

[8] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. SPDZ2k: Efficient MPC mod 2^k for dishonest majority. In *CRYPTO*, 2018.

[9] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*, 2012.

[10] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *CRYPTO*, 2020.

[11] Tore Kasper Frederiksen, Marcel Keller, Emmanuela Orsini, and Peter Scholl. A unified approach to MPC with preprocessing using OT. In *ASIACRYPT*, 2015.

[12] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In *STOC*, 1987.

[13] Gaël Guennebaud and Benoît Jacob. Eigen v3. http://eigen.tuxfamily.org, 2010.

[14] Xiaojie Guo, Kang Yang, Xiao Wang, Wenhao Zhang, Xiang Xie, Jiang Zhang, and Zheli Liu. Half-tree: Halving the cost of tree expansion in COT and DPF. In *EUROCRYPT*, 2023.

[15] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: Secure gpt inference with function secret sharing. In *PETS*, 2024.

[16] Kanav Gupta, Deepak Kumaraswamy, Nishanth Chandran, and Divya Gupta. Llama: A low latency math library for secure inference. In *PETS*, 2022.

[17] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. Orca: Fss-based secure training and inference with gpus. In *IEEE S&P*, 2024.

[18] Hyang-Sook Lee and Jeongeun Park. On the security of multikey homomorphic encryption. Cryptology ePrint Archive, Paper 2019/1082, 2019.

[19] Jingyu Li, Zhicong Huang, Min Zhang, Jian Liu, Cheng Hong, Tao Wei, and Wenguang Chen. PANTHER: Private approximate nearest neighbor search in the single server setting. Cryptology ePrint Archive, Paper 2024/1774, 2024.

[20] Payman Mohassel and Yupeng Zhang. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In *IEEE S&P*, 2017.

[21] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CrypTFlow2: Practical 2-Party Secure Inference. In *CCS*, 2020.

[22] Peter Rindal. cryptoTools. https://github.com/ladnir/cryptoTools, 2023.

[23] Théo Ryffel, David Pointcheval, and Francis Bach. ARIANN: Low-interaction privacy-preserving deep learning via function secret sharing. In *PETS*, 2022.

[24] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. Private approximate nearest neighbor search with sublinear communication. In *IEEE S&P*, 2022.

[25] Siyuan Shang, Xuehui Du, Xiaohan Wang, and Aodi Liu. Private approximate nearest neighbor search for on-chain data based on locality-sensitive hashing. *Future Generation Computer Systems*, 164:107586, 2025.

[26] Kyle Storrier, Adithya Vadapalli, Allan Lyons, and Ryan Henry. Grotto: Screaming fast $(2+1)$-pc for $\mathbb{Z}_{2^n}$ via (2, 2)-dpfs. In *ACM CCS*, 2023.

[27] Sajani Vithana, Martina Cardone, and Flavio P. Calman. Private approximate nearest neighbor search for vector database querying. In *2024 IEEE International Symposium on Information Theory (ISIT)*, pages 3666–3671, 2024.

[28] Sameer Wagh. Pika: Secure Computation using Function Secret Sharing over Rings. *PoPETs*, 2022.

[29] Frank Wang, Catherine Yun, Shafi Goldwasser, Vinod Vaikuntanathan, and Matei Zaharia. Splinter: Practical private queries on public data. In *NSDI*, 2017.

[30] Andrew C. Yao. Protocols for secure computations. In *FOCS*, 1982.

[31] Guy Zyskind, Tobin South, and Alex Pentland. Don't forget private retrieval: distributed private similarity search for large language models, 2023.